Neural Style Transfer Using VGG19

CSCI 611 – Assignment 4
Akshith Macharla

1. Introduction

In this assignment, I implemented neural style transfer based on the paper *Image Style Transfer Using Convolutional Neural Networks* by Gatys et al. The goal of neural style transfer is to generate a new image that preserves the high-level content of one image while transferring the artistic style of another image. In my implementation, I used a pre-trained VGG19 network in PyTorch to extract feature representations for the content image, style image, and target image. The target image was then optimized iteratively so that it matched the content representation of the content image and the style representation of the style image.

The content image used in my experiment was a landscape scene with mountains, trees, a cabin, and a lake reflection. The style image was an artistic painting-style image. The final output successfully preserved the overall scene structure while transferring visible artistic textures and color patterns.

2. Implementation

The implementation follows the general method described by Gatys et al. First, a pre-trained VGG19 convolutional neural network was used as a fixed feature extractor. Intermediate feature maps from selected layers were used to represent content and style.

For content representation, I used the feature map from layer conv4_2. The content loss measures the difference between the target image features and the content image features using mean squared error:

Content Loss = MSE(target content features, content image features)

For style representation, I used Gram matrices computed from multiple convolutional layers. The Gram matrix captures correlations between feature maps and is used to represent texture and style independently of spatial arrangement. The style loss was computed as the weighted sum of mean squared errors between the Gram matrices of the target image and the style image across selected layers.

Style Loss = sum of weighted MSE(target Gram matrix, style Gram matrix)

The total loss combined the content loss and style loss:

Total Loss = content_weight × content_loss + style_weight × style_loss

The target image was initialized and updated through backpropagation using the Adam optimizer. During optimization, the image was modified directly while the VGG19 weights remained fixed. Intermediate results were displayed every 400 iterations to observe the progress of style transfer.

3. Important Code Components

The most important parts of the implementation were:

1. **Feature extraction** using the VGG19 network
   This was used to obtain content and style features from the input images.
2. **Gram matrix computation**
   This converted feature maps into style representations.
3. **Content loss calculation**
   This ensured that the generated image preserved the major structure of the original content image.
4. **Style loss calculation**
   This ensured that the generated image captured the texture, color relationships, and artistic appearance of the style image.
5. **Optimization loop**
   This repeatedly updated the target image to minimize the total loss.

[Insert a short code snippet here for Gram matrix and total loss calculation.]

4. Experiments and Hyperparameter Analysis

To analyze how hyperparameters affect style transfer quality, I performed multiple controlled experiments by changing one important setting at a time while keeping the rest fixed. The baseline settings were:

- content_weight = 1
- style_weight = 1e6
- learning rate = 0.003
- steps = 2000

The following experiments were performed:

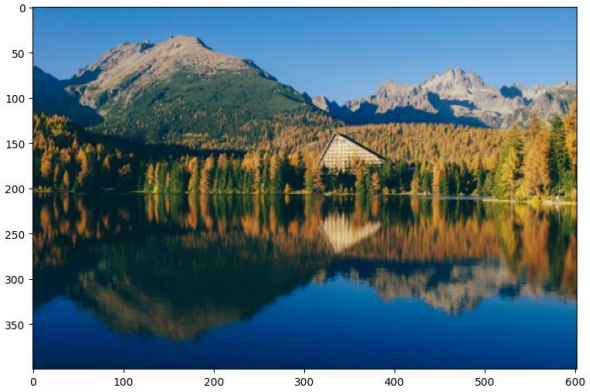
Run	content_weight	style_weight	lr	steps	Main observation
Baseline	1	1e6	0.003	2000	Balanced result
High style	1	5e6	0.003	1000	Stronger style, less content detail
High content	10	1e6	0.003	1000	More content preserved, weaker style
Low steps	1	1e6	0.003	500	Faster but less refined
Low LR	1	1e6	0.001	1000	Slower, smoother optimization

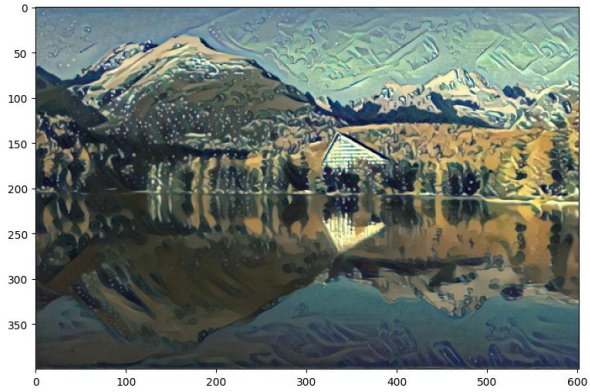
4.1 Baseline Run

The baseline run produced the best balanced result overall. The generated image preserved the mountain, lake, tree line, and cabin structure from the content image while clearly transferring painterly textures and stylized color patterns. The final result looked visually pleasing and showed a good compromise between content preservation and style transfer.

The baseline produced a balanced result. The mountain, lake, and cabin structure were preserved while visible painterly textures were transferred to the target image.

- content_weight = 1
- style_weight = 1e6
- lr = 0.003
- steps = 2000



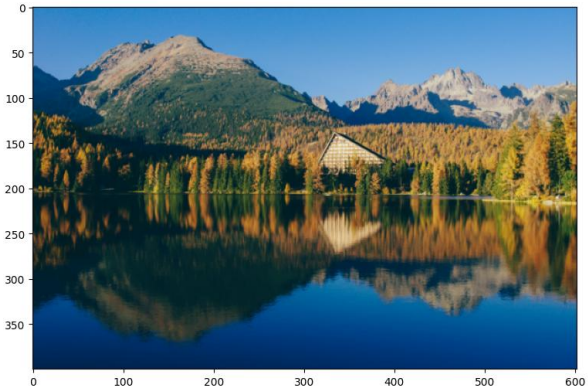


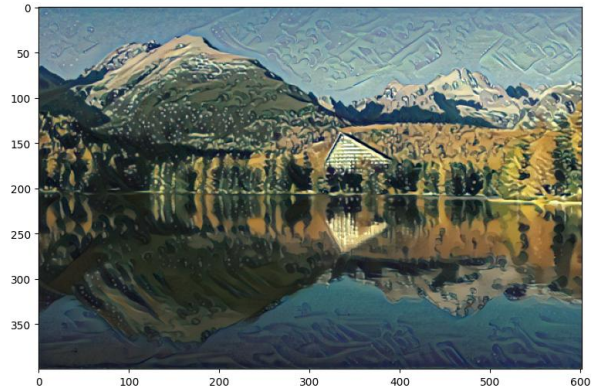
4.2 Higher Style Weight

When the style weight was increased from 1e6 to 5e6, the stylized texture became much stronger. The output image showed more aggressive brush-like patterns and stronger artistic transformation. However, some of the fine details of the original scene became less clear, especially in the reflected lake and the edges of the cabin and mountain structure. This confirms that increasing style weight emphasizes artistic appearance but can reduce content fidelity.

Changed style_weight to 5e6 and steps to 1000

Observation: Increasing style weight made the artistic textures much stronger. However, some of the original scene details became less clear compared to the baseline.



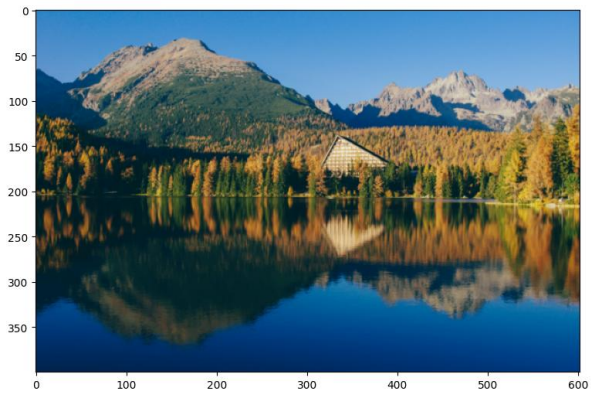
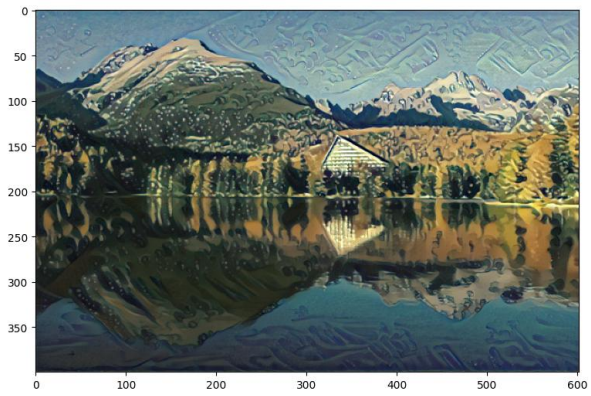


4.3 Higher Content Weight

When the content weight was increased from 1 to 10, the generated image preserved the original scene structure more strongly. The mountains, cabin, trees, and lake reflection remained clearer and closer to the content image. However, the transferred style became weaker and less dramatic. The result looked more like the original photograph with mild stylization instead of a strongly transformed artistic image.

Changed back style_weight to baseline which is original: 1e6 and changed content_weight to 10 keeping steps as 1000

Observation: Increasing content weight preserved the structure of the original image more strongly, but reduced the intensity of the transferred style.

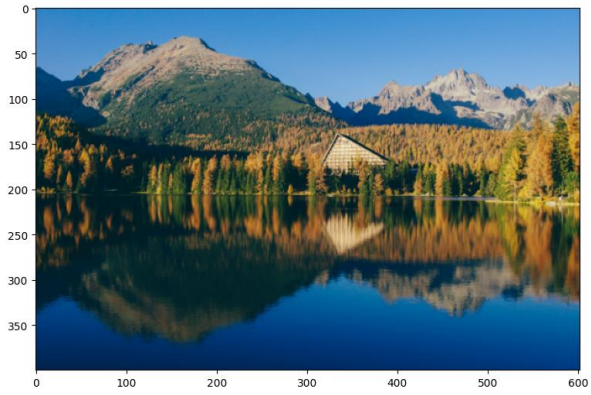
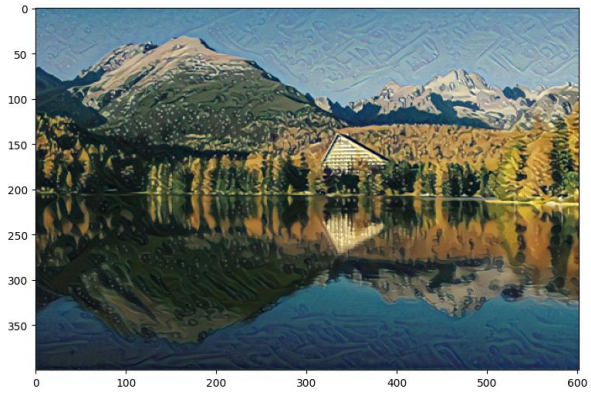
4.4 Fewer Optimization Steps

When the number of optimization steps was reduced from 2000 to 500, runtime decreased significantly. However, the output image appeared less refined, and the artistic style was not fully developed. The textures looked weaker, and the result appeared like an incomplete version of the final stylized image. This shows that sufficient optimization steps are important for convergence and visual quality.

Changed everything back to baseline, content_weight = 1, style_weight=1e6 except making steps =500

Observation:

Reducing the number of optimization steps significantly decreased runtime, but the output looked less refined and the style transfer was not as fully developed.

4.5 Lower Learning Rate

When the learning rate was reduced from 0.003 to 0.001, the optimization progressed more slowly and smoothly. The resulting image was stable, but within the same number of steps the style transfer developed less strongly than in the baseline run. This suggests that a smaller learning rate can produce gradual updates, but may require more iterations to achieve the same visual effect.

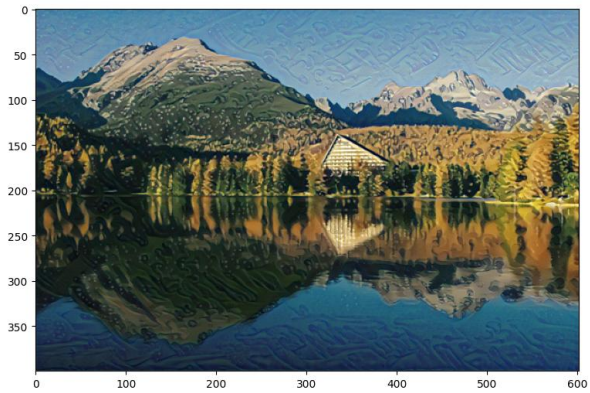
content_weight = baseline

style_weight = baseline

change lr = 0.001

use steps = 1000

observation: A smaller learning rate made the optimization more gradual. The output was more stable, but style transfer developed more slowly within the same number of iterations.



5. Discussion

The experiments show that neural style transfer is highly sensitive to hyperparameter selection. The content weight and style weight directly control the tradeoff between preserving scene structure and applying artistic style. A larger style weight makes the generated image more visually artistic, but can distort or obscure the original content. A larger content weight better preserves the original image layout, but reduces the effect of style transfer.

The number of optimization steps affects how completely the target image converges to the desired balance between content and style. Too few steps produce underdeveloped results, while more steps generally improve refinement at the cost of runtime. The learning rate also influences optimization behavior. A smaller learning rate tends to be smoother and more stable, but it may require more iterations to achieve strong style transfer.

Overall, the baseline setting provided the best balance in my experiments. It preserved the original scene clearly while still producing noticeable artistic transformation. Based on the visual results, moderate hyperparameter values worked better than extreme ones.

6. Conclusion

In this assignment, I successfully implemented neural style transfer using a pre-trained VGG19 network in PyTorch. The generated result demonstrated that deep convolutional features can effectively separate image content from image style and recombine them into a new image. My implementation used content loss from higher-level VGG features and style loss from Gram matrices across multiple layers.

Through hyperparameter experiments, I observed that increasing style weight strengthens artistic texture, increasing content weight preserves scene structure, reducing steps lowers quality, and reducing learning rate slows the optimization process. Among all tested settings, the baseline produced the most balanced and visually appealing output.

This assignment helped me understand how convolutional neural networks can be used not only for classification, but also for image synthesis and optimization-based visual generation tasks.

Run	content_weight	style_weight	lr	steps	Main observation
Baseline	**1**	**1e6**	**0.003**	**2000**	**Balanced result**
High style	**1**	**5e6**	**0.003**	**1000**	**Stronger style, less content detail**
High content	**10**	**1e6**	**0.003**	**1000**	**More content preserved, weaker style**
Low steps	**1**	**1e6**	**0.003**	**500**	**Faster but less refined**
Low LR	**1**	**1e6**	**0.001**	**1000**	**Slower, smoother optimization**